AI ASSISTANT CODING

ASSIGNMENT-13.3

Name: T. Swetha

Roll no: 2303a51317

Batch: 05

Task 1: Refactoring – Removing Code Duplication

Objective: To eliminate repeated logic by extracting reusable functions.

Prompt: Refactor the Python code by removing repeated lines.
Create one function to find the area and perimeter of a rectangle using length and width**.**

Refactor the Python code by removing repeated lines.
Create one function to find the area and perimeter of a rectangle using length and width.

Code:

```python
def rectangle_metrics(length: float, width: float) -> tuple[float, float]:
    """Return (area, perimeter) for a rectangle.
    Args:
        length: The length of the rectangle.
        width: The width of the rectangle.
    Returns:
        A tuple containing (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
if __name__ == "__main__":
    rectangles = [(5, 10), (7, 12), (10, 15)]
    for length, width in rectangles:
        area, perimeter = rectangle_metrics(length, width)
        print("Area of Rectangle:", area)
        print("Perimeter of Rectangle:", perimeter)
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Programs/Python/Python313/python.exe c:/Users/thaku/One
Drive/Desktop/Training/attendance_frontend/13.3.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Code Explanation:

- The program defines a function rectangle_metrics(length, width) to calculate the area and perimeter of a rectangle.
- It uses type hints (float) to indicate that the length and width should be floating-point numbers and returns a tuple containing area and perimeter.
- A docstring is included to describe the function's purpose, its parameters (length, width), and the values it returns.
- Inside the function, the area is calculated using the formula area = length × width.
- The perimeter is calculated using the formula perimeter = 2 × (length + width).
- The function returns both values as a tuple (area, perimeter).
- The block if __name__ == "__main__": ensures that the main code runs only when the script is executed directly.
- A list named rectangles stores multiple rectangle dimensions as tuples (length, width).
- A for loop iterates through each rectangle, calls the rectangle_metrics() function, and retrieves the calculated values.
- Finally, the program prints the area and perimeter for each rectangle.

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective: To improve performance by replacing inefficient nested loops with optimized structures.

Prompt: Analyse the following Python code and refactor it to improve performance by removing inefficient nested loops. Use optimized structures such as sets or list comprehensions for faster lookups. Also explain the performance improvement before and after refactoring in simple terms.

> Analyze the following Python code and refactor it to improve performance by removing inefficient nested loops. Use optimized structures such as sets or list comprehensions for faster lookups. Also explain the performance improvement before and after refactoring in simple terms

Code:

```python
def rectangle_metrics(length: float, width: float) -> tuple[float, float]:
    """Return (area, perimeter) for a rectangle.
    Args:
        length: The length of the rectangle.
        width: The width of the rectangle.
    Returns:
        A tuple containing (area, perimeter).
    """
def report_name_presence(names: list[str], search_names: list[str]) -> None:
    """Print whether each searched name is present in the provided list.
    Uses a set for O(1) lookups, which is much faster than nested loops
    when the list gets larger.
    """
    name_set = set(names)
    for s in search_names:
        if s in name_set:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")
if __name__ == "__main__":
    names = ["Alice", "Bob", "Charlie", "David"]
    search_names = ["Charlie", "Eve", "Bob"]
    report_name_presence(names, search_names)
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Programs/Python/Python313/python.exe c:/Users/thaku/OneDri
ve/Desktop/Training/attendance_frontend/13.3.py
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Code Explanation:

- The legacy code checks whether elements exist in a list using nested loops, which increases the number of comparisons.
- For each name in search_
- names, the program iterates through the entire names list, making the process slower for large datasets.
- The optimized version converts the list names into a set using set(names) for faster searching.
- Set membership checking (in) is much faster because sets use hashing, allowing near constant-time lookup.
- This optimization removes the inner loop, simplifying the code and improving readability.
- The time complexity improves from O(n × m) in the nested loop approach to O(n + m) using a set lookup.
- As a result, the refactored code runs faster and is more efficient for large lists.

Task 3: Refactoring – Extracting Reusable Functions

Objective: To modularize code by extracting calculations into reusable functions.

Prompt: Refactor the following Python legacy code by creating a reusable function named calculate_total(price). The function should calculate tax (18%) and return the total price. Improve the code structure, remove repetition, and add proper documentation (docstring) explaining the function.

```
Refactor the following Python legacy code by creating a reusable function named
calculate_total(price). The function should calculate tax (18%) and return the total
price. Improve the code structure, remove repetition, and add proper
documentation (docstring) explaining the function.
```

Code:

```python
def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """Calculate total price including tax.
    The tax is computed as `price * tax_rate`.
    Args:
        price: Base price.
        tax_rate: Tax rate to apply (default 18%).
    Returns:
        Total price after adding tax.
    """
    return price + price * tax_rate
if __name__ == "__main__":
    for price in (250, 500):
        total = calculate_total(price)
        print("Total Price:", total)
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Programs/Python/Python31
3/python.exe c:/Users/thaku/OneDrive/Desktop/Training/attendance_frontend/13.3.py
Total Price: 295.0
Total Price: 590.0
```

Code Explanation:

- The program defines a function **calculate_total(price, tax_rate=0.18)** to calculate the **total price including tax**.
- price: float indicates that the **price should be a floating-point number**, and tax_rate has a **default value of 0.18 (18%)**.

- A **docstring** is included to explain the purpose of the function, its parameters (price, tax_rate), and the value it returns.
- Inside the function, the **total price is calculated using the formula**: **Total = price + (price × tax_rate)**.
- The function **returns the total price after adding tax**.
- The block **if __name__ == "__main__":** ensures that the code runs only when the script is executed directly.
- A **for loop** iterates through two prices (250, 500).
- For each price, the function **calculate_total()** is called to compute the total amount.
- Finally, the program **prints the total price including tax** for each value.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective: To improve maintainability by replacing magic numbers with named constants.

Prompt: Analyze the following Python code and refactor it by replacing hardcoded values (magic numbers) with named constants. Use constants such as PI and RADIUS to make the code cleaner and easier to maintain.

```
Analyze the following Python code and refactor it by replacing hardcoded values
(magic numbers) with named constants. Use constants such as PI and RADIUS to
make the code cleaner and easier to maintain
```

Code:

```python
PI = 3.14159
RADIUS = 7
def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """Calculate total price including tax.
    The tax is computed as `price * tax_rate`.
    Args:
        price: Base price.
        tax_rate: Tax rate to apply (default 18%).
    Returns:
        Total price after adding tax.
    """
    return price + price * tax_rate
if __name__ == "__main__":
    area_of_circle = PI * (RADIUS ** 2)
    circumference = 2 * PI * RADIUS

    print("Area of Circle:", area_of_circle)
    print("Circumference of Circle:", circumference)
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Programs/Python/Python31
3/python.exe c:/Users/thaku/OneDrive/Desktop/Training/attendance_frontend/13.3.py
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Code Explanation:

- Two constants are defined at the beginning:
- PI = 3.14159 → represents the value of π used in circle calculations.
- RADIUS = 7 → represents the radius of the circle.
- Using constants instead of hardcoded values improves code readability and makes it easier to update values later.
- A function calculate_total(price, tax_rate=0.18) is defined to calculate the total price including tax.
- The function includes a docstring that explains the purpose of the function, its parameters (price, tax_rate), and the return value.
- Inside the function, the total price is calculated using the formula:
  total = price + (price × tax_rate).
- The block if __name__ == "__main__": ensures that the code runs only when the script is executed directly.
- The area of the circle is calculated using the formula:
  Area = PI × RADIUS².
- The circumference of the circle is calculated using the formula:
  Circumference = 2 × PI × RADIUS.

Task 5: Refactoring – Improving Variable Naming and Readability

Objective: To enhance readability using descriptive variable names and comments.

Prompt: Refactor the following Python code by improving readability. Replace unclear variable names with descriptive variable names and add inline comments explaining the logic. Ensure the program produces the same output as the original code.

```
Refactor the following Python code by improving readability. Replace unclear
variable names with descriptive variable names and add inline comments
explaining the logic. Ensure the program produces the same output as the original
code
```

Code:

```python
def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """Calculate total price including tax.
    The tax is computed as `price * tax_rate`.
    Args:
        price: Base price.
        tax_rate: Tax rate to apply (default 18%).
    Returns:
        Total price after adding tax.
    """
    return price + price * tax_rate
if __name__ == "__main__":
    # Calculate triangle area using base and height
    # (same result as `c = a * b / 2` with clearer names)
    base_length = 10
    height = 20
    triangle_area = base_length * height / 2
    print(triangle_area)
```

Output:

```
3/python.exe c:/Users/thaku/OneDrive/Desktop/Training/attendance_frontend/13.3.py
100.0
```

Code explanation:

- The program assigns the value 10 to variable a and 20 to variable b.
- It calculates a value using the formula c = a * b / 2.
- This calculation multiplies a and b first, then divides the result by 2.
- The result of this calculation is stored in the variable c.
- Finally, the program prints the value of c using the print() function.
- With the given values (a = 10, b = 20), the output will be:
  100.0